VORONOI-BASED INDOOR ROOM SEARCH AND RESCUE

Meshaal. Mouawad
Department of Electrical and Computer Engineering, Mississippi State University
This technical report is to process of my research during the Summer, 2021.
Technical Report

***Abstract*—In this project, we investigate of autonomous robots for mapping, navigation, with the shortest path planning to the target. We implemented Voronoi Diagram for path planning such as the Global Path and avoiding the obstacles, we generated local points (multiway-points) and applied Travel Salesman Problem (TSP) Algorithms to find the optimal path (shortest path planning) to connect the waypoints by searching the closest edge of the voronoi diagram, we then applied the Dynamic Window Approach method (DWA) to allow the robot to navigate locally based on the TSP and voronoi diagram paths. The robot was given an assignment to navigate from the start point (source) to the end point with local points (targets). Then the robot has to complete the assignment as the followings from the start point find the shortest path, go to the local points (the targets), with avoiding the obstacles.**

Index Terms:

I. PROJECT OUTLINE

SEARCH and rescue is the final goal of this research project. Assume we have a disaster, and we want to search people whom were injured inside a building and we want the robot to navigate and search inside the rooms. To do this we need to find the path planning, determine the targets, find the optimal path for the target with shortest distant, then navigate locally to the targets. This can be categorized as four stages as the followings: Environmental Exploration, Multi-Goal Path Planning, Local Navigator, and 2D Grid Mapping.

In the Environmental Exploration stage, we constructed two kinds of Voronoi Diagram maps: Voronoi Diagram, and Generalized Voronoi Diagram (GVD) as follows as it shows in fig. 1. and fig. 2.

In the Multi-goal path planning, we used the Voronoi Diagram (GVD) maps with for TSP (Travelling salesman problem) algorithms to connect some goals/waypoints that we need to explore, rescue, and reach. Running your TSP algorithm to connect some points such as the green circles P_n in Fig. 3. TSP can connect waypoints, $P_0, P_1, P_2 \dots P_n$ (injured persons – we have known their GPS coordinates such as the local points in Table. 1) in certain sequence to minimize the total traveling distance. The result would be the optimal path, the green path in Fig. 3, for the targets (local points).

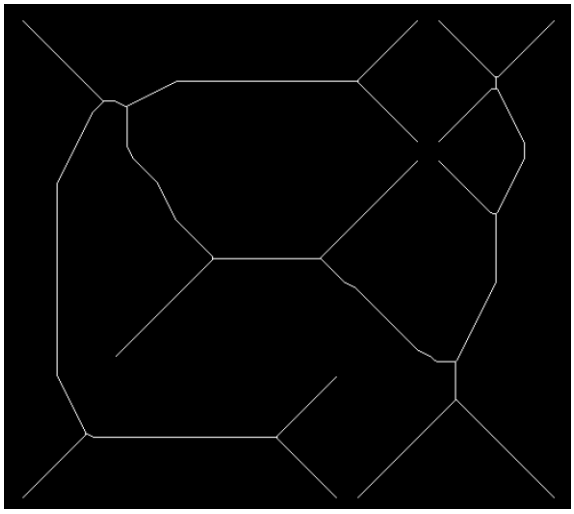


Fig. 1. Generalized Voronoi Diagram (GVD

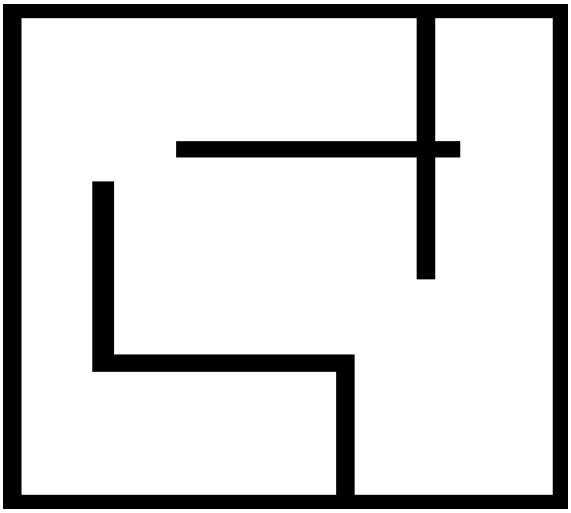


Fig. 4. The original map we used in the project.

Meshaal Mouawad, PhD Student, Mississippi State University, Starkville, MS 39759 USA (e-mail: mm4922@msstate.edu). *

TABLE I
THE LOCAL POINTS FOR TSP

Local locations	Local Points	X	Y
0	P_0	35	33
1	P_1	258	73
2	P_2	257	230

TSP local points as we chose for the targets

The red points are the Voronoi diagram points where number 1 is the start points and number 25 is the end point.

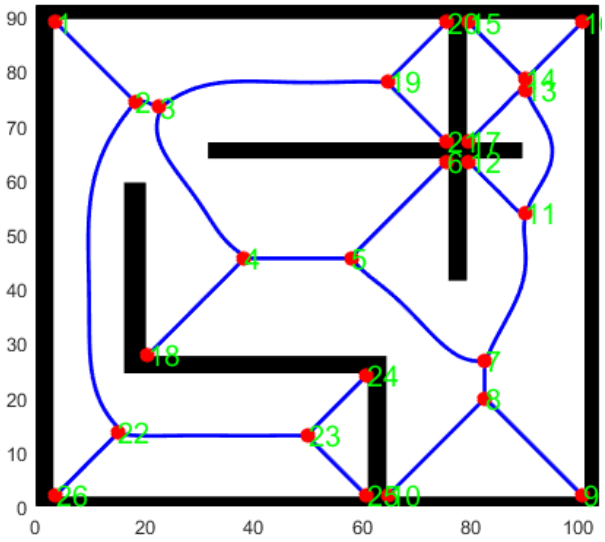


Fig. 2. Voronoi Diagram with the path planning

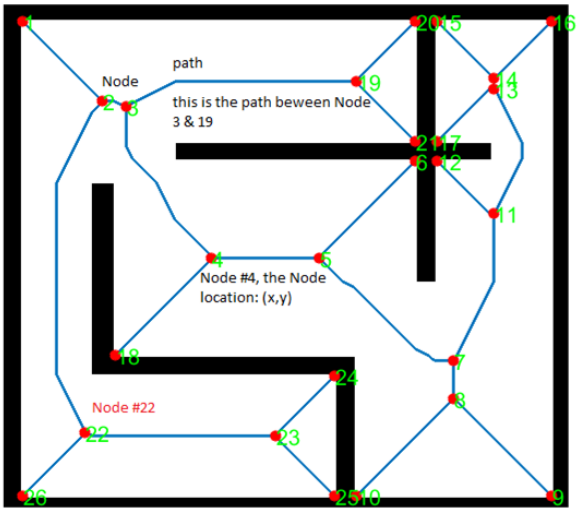


Fig. 6. The Voronoi Diagram specifications

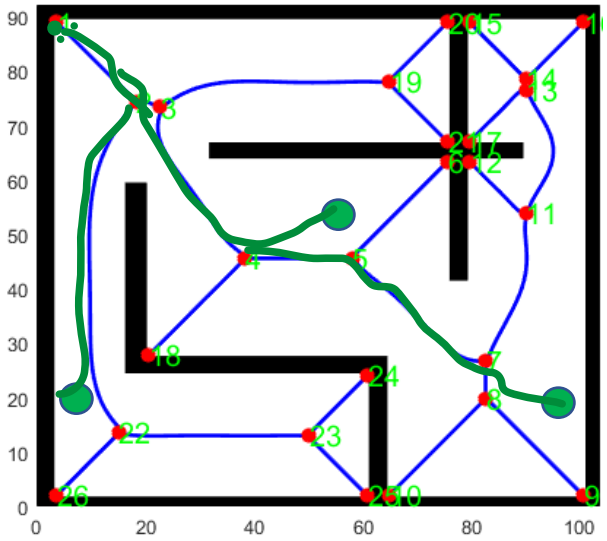
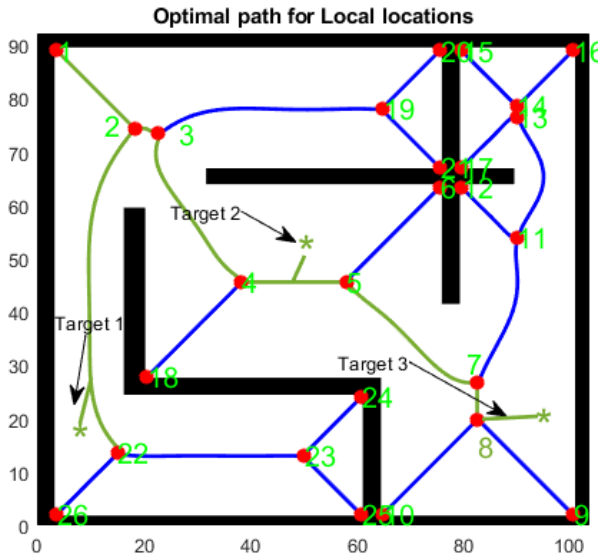


Fig. 3 the expected optimal path using TSP (Travelling salesman problem) algorithms. the blue line is the path planning for the Voronoi Diagram, the green path is the optimal path for the targets (local points), and the green circles are the targets.



Once sequence has been determined, robot needs to carry out path planning (navigation).
• Robot will perform navigation/path planning.
• Build up map (Mapping).

The local navigator we implemented in this project is the DWA (Dynamic Window Approach) method to walk a mobile robot from starting point, the red point number 1, to goal, the P_n green points, along the edges of built GVD, the blue path. This task significantly to using DWA to navigate the robot to
connect multiple goals depicted by the green points such that the green path in Fig. 3.

II. THE PROJECT STAGES

1) Environmental Exploration

We constructed the Voronoi Diagram using the map in Fig. 4. The map black lines are the walls

(obstacles). The robot can only move in the empty area in the map and avoid any obstacles such as the walls. To find the global path, we used the Voronoi Diagram method as it shows in Fig. 2. In this stage we must specified the robot radius, the obstacles, the board. The radius of the robot, is used to compute configuration free space as it shows in Fig. 5. in case if the robot's radius is big, the path in the graph will be disconnected which means it can not pass through a narrow corridor or door in the map due to the collision's expectations.

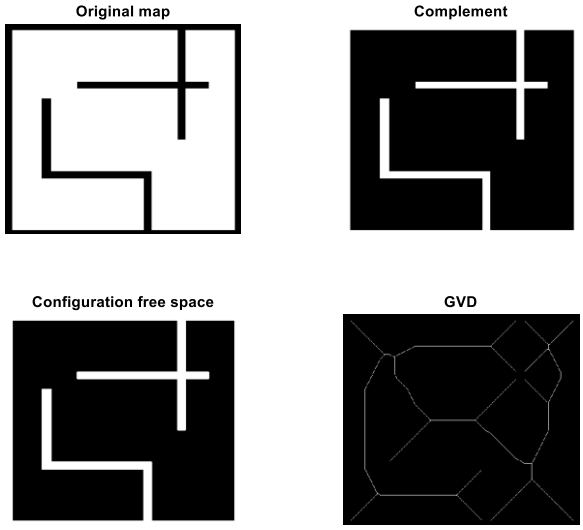


Fig. 5. The free space of the map in Voronoi diagram.

2) Multi-goal path planning

We chose local points P_n such as the points in Table.1 as a target (waypoints) similar to those in Fig. 3 by choosing its locations on the generated Voronoi diagram by adding its coordinates to the map. This stage we calculated the path length matrix using the equation

$$\sqrt{(p_{(i+1,1)} - p_{(i,1)})^2 + (p_{(i+1,2)} - p_{(i,2)})^2} \quad (1)$$

Where P is the path between the nodes

Then we combined all points in a map with the targets to find the optimal path to the local points using TSP. We formulated TSP algorithms by an integer linear programming and solved it by the Miller–Tucker–Zemlin (MTZ) formulation [2]. For time optimization, we used Goruobi modules as a solver for TSP.

I have not completed adding the targets

3) Local navigator

DWA method will breakdown the optimal path, the green path specifically between waypoints, to multiple points that DWA can navigate locally.

Here is the last result where the robot will navigate to the local points and achieve the goal. We try to search the closest edge of the Voronoi diagram.

Not completed yet

4) Mapping

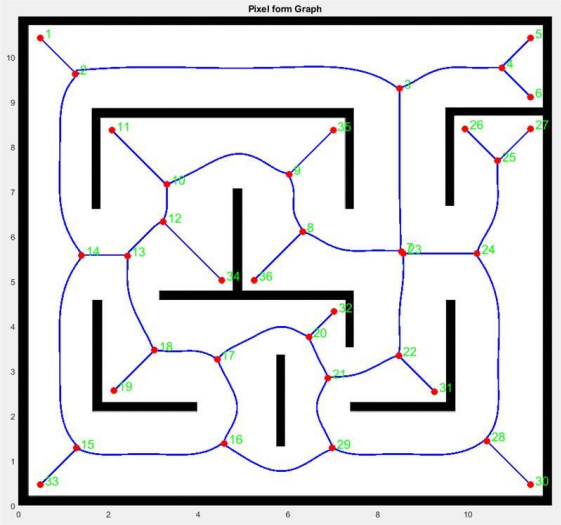
III. CONCLUSION

References

Fig. 6. Domenistrates the Voronoi Diagram.

APPENDIX. I

From VD, we have this kind of result, 36 nodes in the graph



You first need to get the "Node distance matrix", which means the node to other nodes path length (not euclidean distance) .

The path can be found in your code from the variable "EdgeLabel" or "GRAPH"

For example, double click the "74*2 double", which means the path from node "1" to node"2". Then you will see the path x, y coordinate from node 1 to node 2

```
GRAPH: CONTAINS NODES AND PATH BETWEEN
NODES(IN PIXEL)
from : the corresponding start node
to :   the corresponding end node

QUADGRAPH: CONTAINS THE GRAPH IN THE FORM
OF QUADRATIC FUNCTION FOR EACH EDGE
number_nodes: number of nodes in the
constructed graph
node_list: location of the nodes
edge_matrix: Adjacency matrix of the
graph
path_matrix: The Path between all the
nodes
w_s :  size of the work space
Pol_coefs:  coefficent of the quadratic
form of the edges
```

Variables - GRAPH

GRAPH

1x25 struct with 3 fields

Fields	path	from	To
1	74x2 double	1	2
2	22x2 double	2	3
3	139x2 double	3	4
4	99x2 double	4	5
5	89x2 double	5	6
6	124x2 double	5	7
7	35x2 double	7	8
8	90x2 double	8	9
9	90x2 double	8	10
10	136x2 double	7	11
11	52x2 double	11	12
12	116x2 double	11	13
13	10x2 double	13	14
14	53x2 double	14	15
15	54x2 double	14	16
16	53x2 double	13	17
17	90x2 double	4	18
18	212x2 double	3	19
19	55x2 double	19	20
20	56x2 double	19	21
21	305x2 double	2	22

This contains the *path* and you need to calculate the distance by adding the length step by step.

```
pathstep =  size(path,1);
 pathlength = 0;
for i = 1: pathnode-1
 temp = sqrt((path(i+1,1)-
path(i,1))^2+(path(i+1,2)-
path(i,2))^2);
pathlength = pathlength + temp;
end pathlength
```

The above is a sample to calculate the path length.

Table.2 The Node List

Node Number	X	Y
1	3.6	89
2	18.2	74.4

3	22.6	73.4
4	38.2	45.6
5	58	45.6
6	75.6	63.4
7	82.6	26.8
8	82.6	19.8
9	100.6	2
10	64.8	2
11	90	53.8
12	79.6	63.4
13	90	76.6
14	90	78.6
15	79.6	89
16	100.6	89
17	79.6	67
18	20.6	27.8
19	64.8	78
20	75.6	89
21	75.6	67
22	15	13.6
23	50	13
24	60.8	24
25	60.8	2
26	3.6	2

Table.2, contains the coordinalte (x,y) for each node in the voronoi diagram. It has 26 Nodes, with x & y locations. For example Node 1 location (3.6,89), Node 2 (18.2,74.4) … etc

Variables - GRAPH

GRAPH

1x25 struct with 3 fields

Fields	path	from	To
1	74x2 double	1	2
2	22x2 double	2	3
3	139x2 double	3	4
4	99x2 double	4	5

Here is the Path coordinate (x,y). from Node 1 to Node 2 the path between them has this location using Eq. 1. And `GRAPH(1).path(1)` which is in Fig. 8, For node 1 to node 2, the path (x,y) corrdinate start from (16,18), (17,19) …to the end at (89, 91)

GRAPH(1).path

GRAPH(1).path

	1	2
1	16	18
2	17	19
3	18	20
4	19	21
5	20	22
6	21	23
7	22	24
8	23	25
9	24	26
10	25	27
11	26	28
12	27	29
13	28	30
14	29	31
15	30	32
16	31	33
17	32	34
18	33	35
19	34	36
20	35	37
21	36	38

Fig. 8. Path in Graph

$$\frac{\sqrt{(p_{(i+1,1)} - p_{(i,1)})^2 + (p_{(i+1,2)} - p_{(i,2)})^2}}{\sqrt{(path_{2,1} - path_{1,1})^2 + (path_{1,2} - path_{2,2})^2}}$$

Comparing the equation and the result from the MATLAB

$$\sqrt{(GRAPH(1).path(2,1) - GRAPH(1).path(1,1))^2 + ((GRAPH(1).path(1,2) - GRAPH(1).path(2,2))^2}$$

GRAPH(1).path(2,1) = 17
GRAPH(1).path(1,1) = 16
GRAPH(1).path(2,2) = 19
GRAPH(1).path(1,2) = 18

$$\sqrt{(17-16)^2 + (19-18)^2} = 1.4142$$

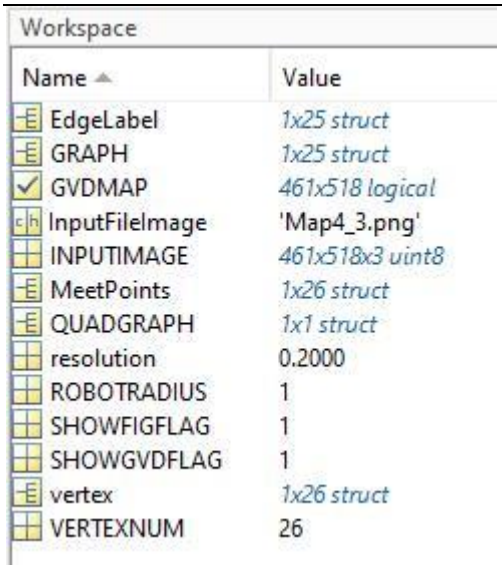
Since Size (GRAPH(1).path) = 74
$74 \times 1.4142 = 103.23$ therfore the length between node 1 and node 2 is 103.23

Then we do the same with path 2, GRAPH(2).patth
Then the length to path 3 = path from node 1 to node 2+ the path from node 2 to node 3

After we calculated the path length now we need to find the distant matrix

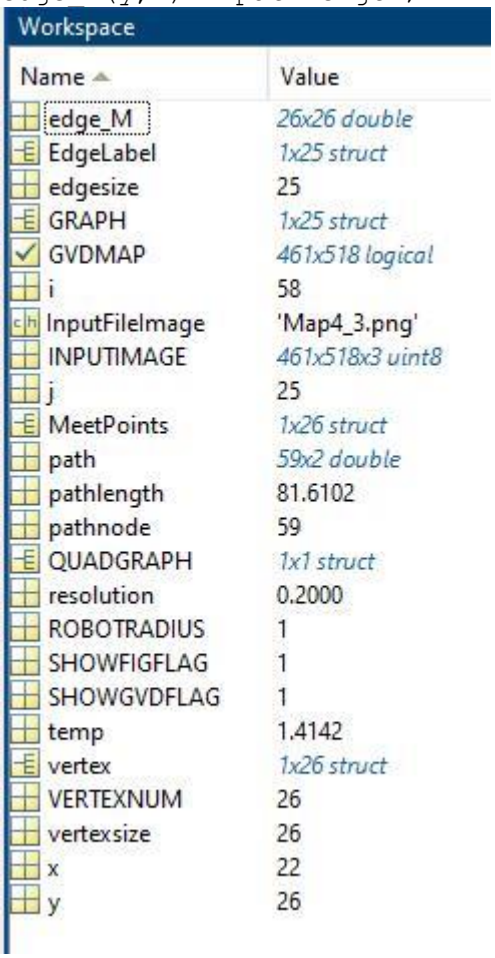
Open `QUDARATICGRAPH`
from VD.
Then open
Create matrix `26 * 26`

```
path = GRAPH(1).path
pathnode =  size(path,1);
pathlength = 0;
for i = 1: pathnode-1
temp = sqrt((path(i+1,1)-
path(i,1))^2+(path(i+1,2)-
path(i,2))^2);
pathlength = pathlength + temp;
end
pathlength
```



can you copy and paste theses code at the end of `MainGraphMaker.m`

```
edgesize = size(GRAPH,2);
vertexsize = size(vertex,2);
edge_M = zeros(vertexsize,vertexsize);
for j = 1 : edgesize
path = GRAPH(j).path;
pathnode = size(path,1);
pathlength = 0;
for i = 1: pathnode-1
temp = sqrt((path(i+1,1)-
path(i,1))^2+(path(i+1,2)-
path(i,2))^2);
pathlength = pathlength + temp;
end
x = GRAPH(j).from;
y = GRAPH(j).To;
edge_M(x,y) = pathlength;
edge_M(y,x) = pathlength;
```



And in your workspace, `edge_M` is the edge matrix for the map

You then need to add a few targets that are not in the obstacle, such as 4 targets. Then you need to get a 31*31 distance matrix. Please code from the line 146.

[6:29 PM] Lei, Tingjun
Can you remember the code in the Edge_matrix is the code I give you last night. That's what I expect in the code

[6:29 PM] Lei, Tingjun

Then you need based on the result of the edge matrix to get the distance matrix

[6:30 PM] Lei, Tingjun
as I did in this code

[6:30 PM] Lei, Tingjun
So last night, I actually help you to finish the first step and expect you to finish the second step

[6:32 PM] Lei, Tingjun
Now I think you get the final code of the first two steps and I hope you to finish the third step "Distance matrix with targets"
(1 liked)

[6:32 PM] Lei, Tingjun
In this part, the code you need has the following functions.
First, find the node near the target point.
Second, get the distance from the target point to nearby points.
Third, update the whole distance matrix.
Edited